



Instituto Tecnológico de las Américas

CYBEROPS

Análisis de Aplicación, Tráfico Local y Encodings

Estudiante

Alison Domeirys Caines Bautista

Matrícula

2024-1113

Fecha

12 de junio de 2026

Maestro

Jonathan Rondón

Asignatura

Seguridad de redes

Institución

ITLA – Instituto Tecnológico de las Américas

Video: <https://youtu.be/XjdAtFpYwal>

Github: <https://github.com/alisoncrespita>

1. Objetivo de la Práctica

El objetivo de esta práctica es que el estudiante analice una aplicación y descubra funcionalidades ocultas relacionadas con redes, servicios locales, encodings y validación remota, aplicando técnicas básicas de reversing, análisis de tráfico y enumeración de servicios.

Las actividades principales fueron:

- Ejecutar y analizar el comportamiento del programa.
- Identificar comunicaciones de red generadas por la aplicación.
- Descubrir servicios locales abiertos.
- Interactuar con dichos servicios para obtener pistas.
- Decodificar correctamente la información obtenida.
- Comprender el mecanismo completo de validación de la contraseña.

2. Herramientas Utilizadas

- WinRAR – descompresión del archivo adjunto.
- Wireshark 4.6 – captura y análisis de tráfico de red.
- dnSpy v6.1.8 – decompilación y reversing del binario .NET.
- ncat – conexión al servicio local de pistas.
- netstat – enumeración de puertos abiertos.
- Máquina virtual Windows 10 x64 sobre VMware Workstation.

3. Flujo Completo del Programa – Paso a Paso

3.1 Preparación del Entorno

Se descargó el archivo adjunto (Clave-es-Itla1234) que contenía la aplicación WinFormsApp1 compilada en .NET 8.0 para Windows. En el escritorio se contaba con las herramientas necesarias: dnSpy, Wireshark, WinRAR y las carpetas del reto.



Figura 1 – Escritorio con las herramientas disponibles (dnSpy, Wireshark, WinRAR, carpeta del reto)

3.2 Ejecución Inicial del Programa

Al ejecutar WinFormsApp1.exe, se presentó una ventana titulada "CyberOps" solicitando una contraseña. La carpeta net8.0-windows contiene: WinFormsApp1.deps.json, WinFormsApp1.dll, WinFormsApp1.exe (148 KB), WinFormsApp1.pdb y WinFormsApp1.runtimeconfig.json.

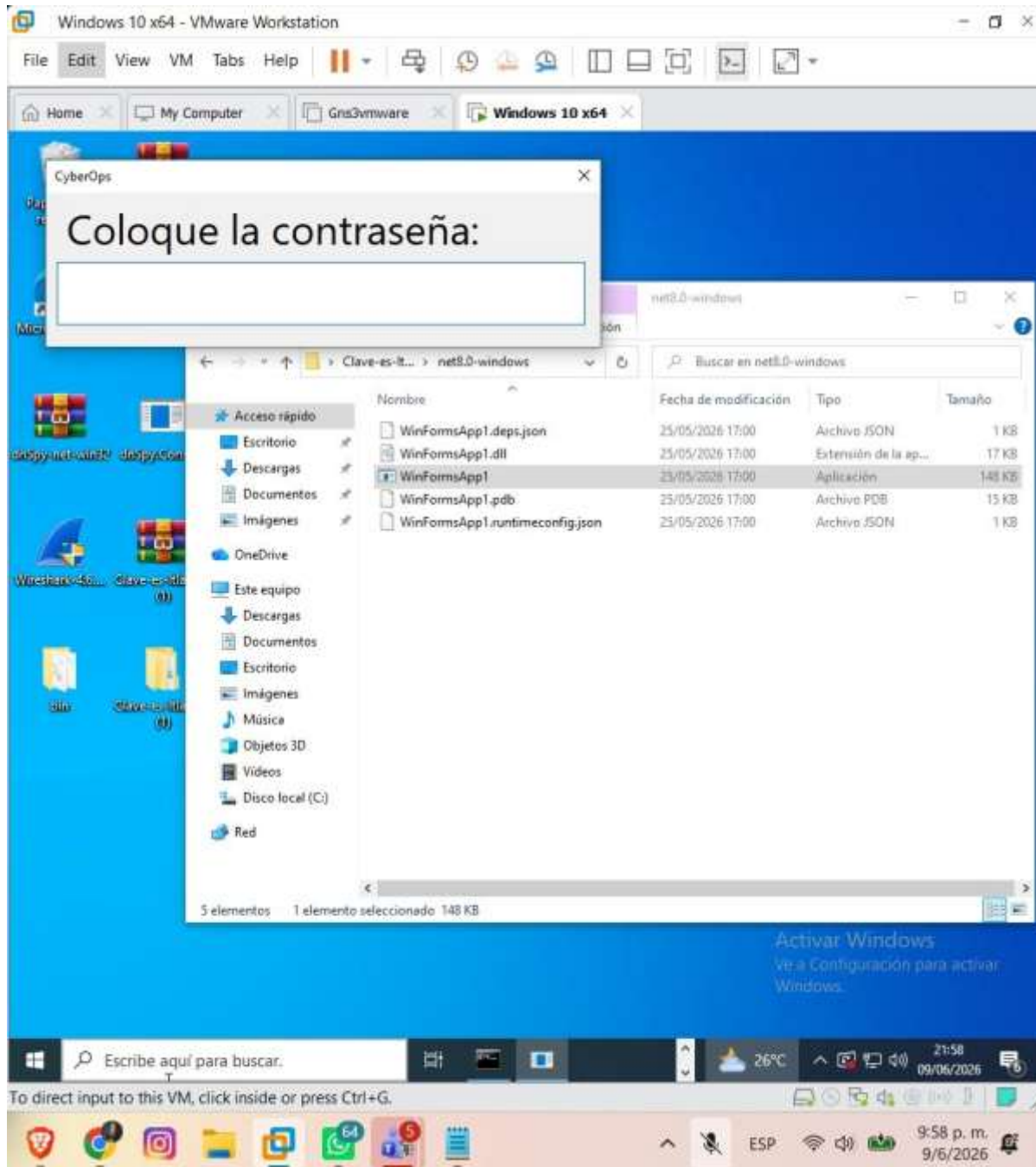


Figura 2 – Contenido de la carpeta net8.0-windows con el ejecutable seleccionado

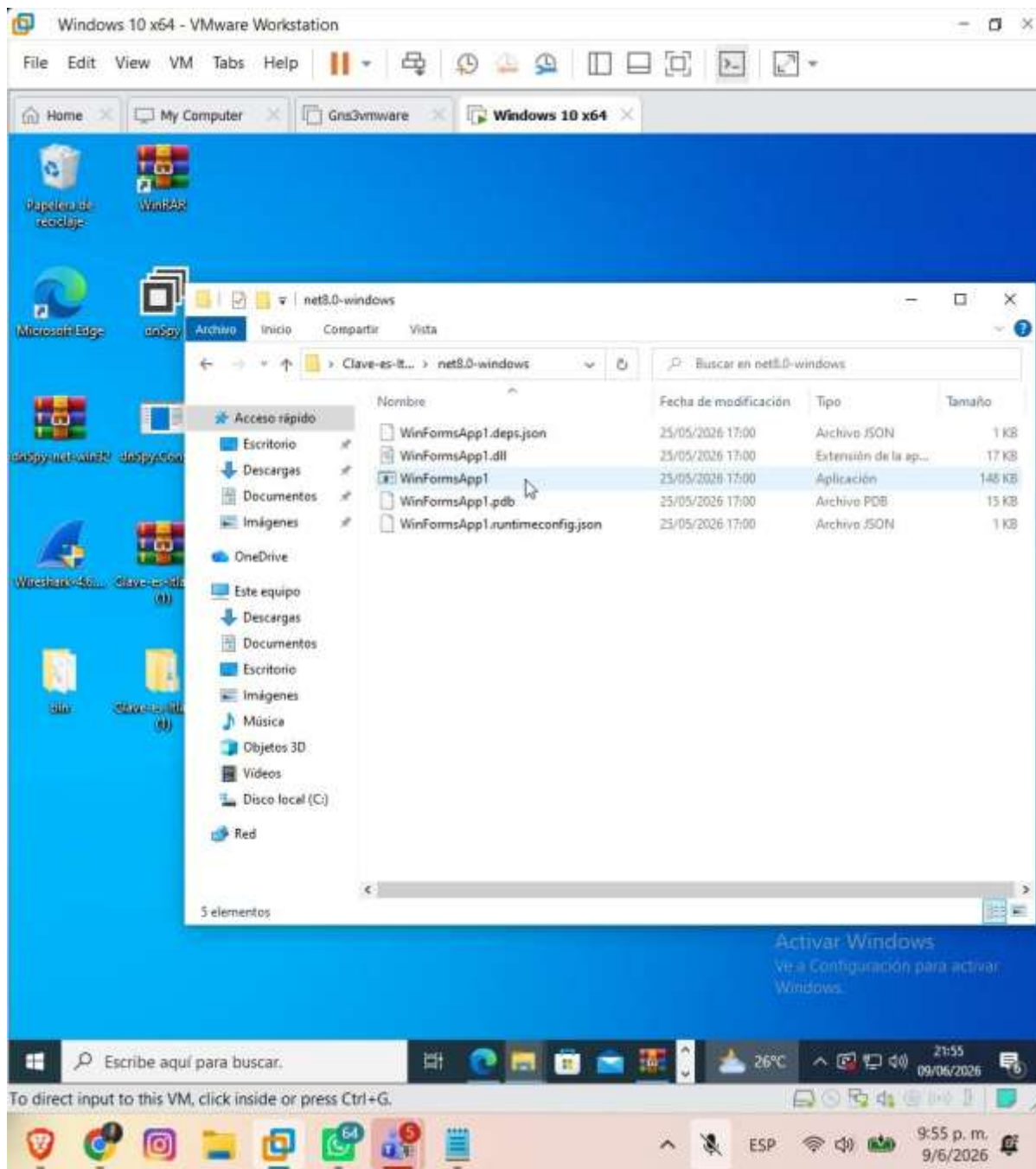


Figura 3 – Ventana "CyberOps" solicitando contraseña al ejecutar WinFormsApp1

3.3 Captura de Tráfico con Wireshark

Se inició Wireshark en la interfaz Ethernet0 antes de ejecutar el programa, para observar las comunicaciones generadas. La aplicación generó tráfico ICMP (ping) hacia 8.8.8.8 (Google DNS), y tráfico TCP hacia la IP 52.85.78.90 por el puerto 443 (HTTPS). Esto indica que el programa verifica conectividad a internet mediante ICMP y luego realiza una conexión HTTPS.

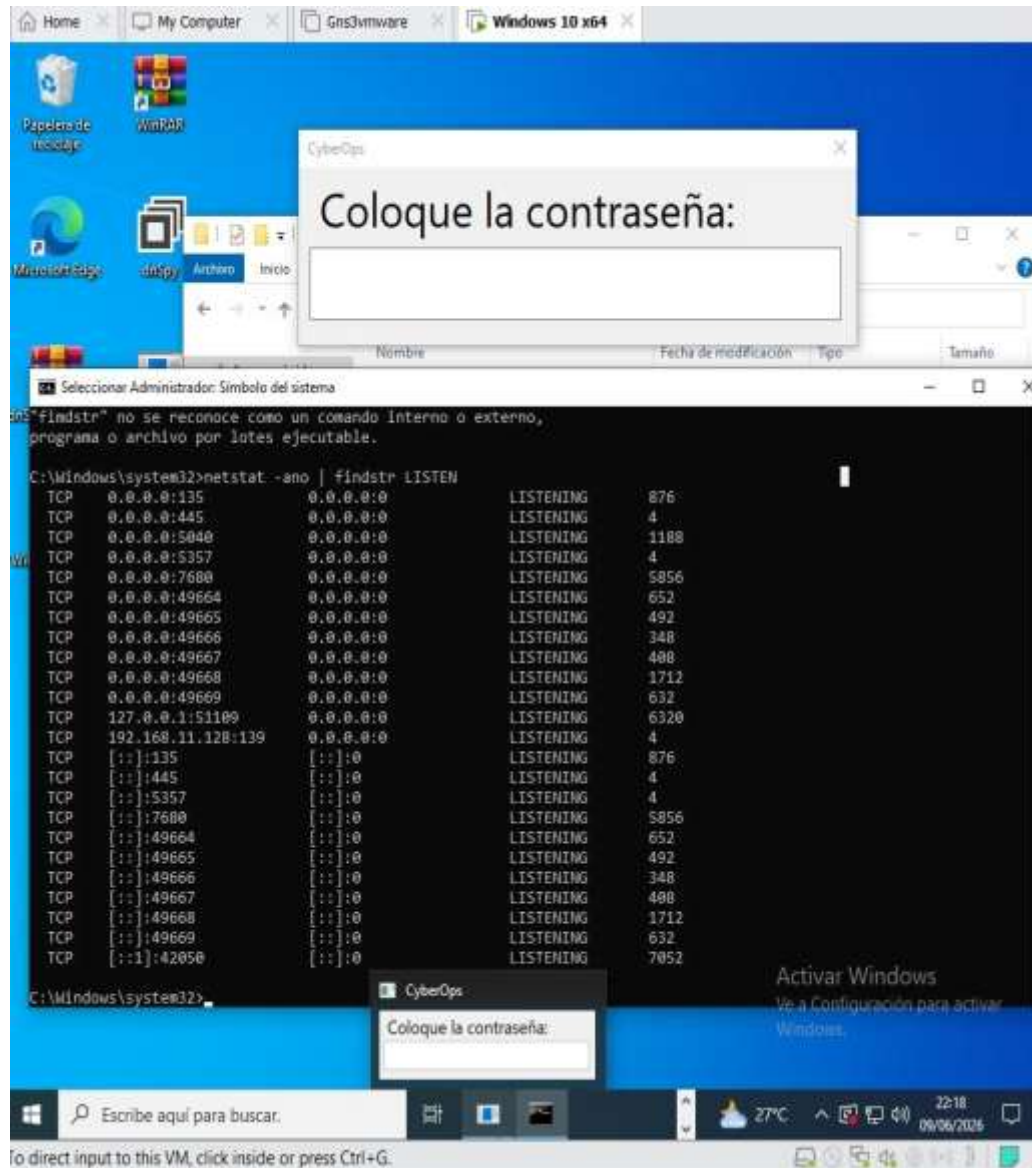


Figura 4 – Captura Wireshark mostrando tráfico ICMP hacia 8.8.8.8 y TCP hacia 52.85.78.90:443

3.4 Enumeración de Puertos con netstat

Se abrió una consola de comandos como administrador y se ejecutó el comando "netstat -ano | findstr LISTEN" para identificar los puertos en escucha abiertos por la aplicación.

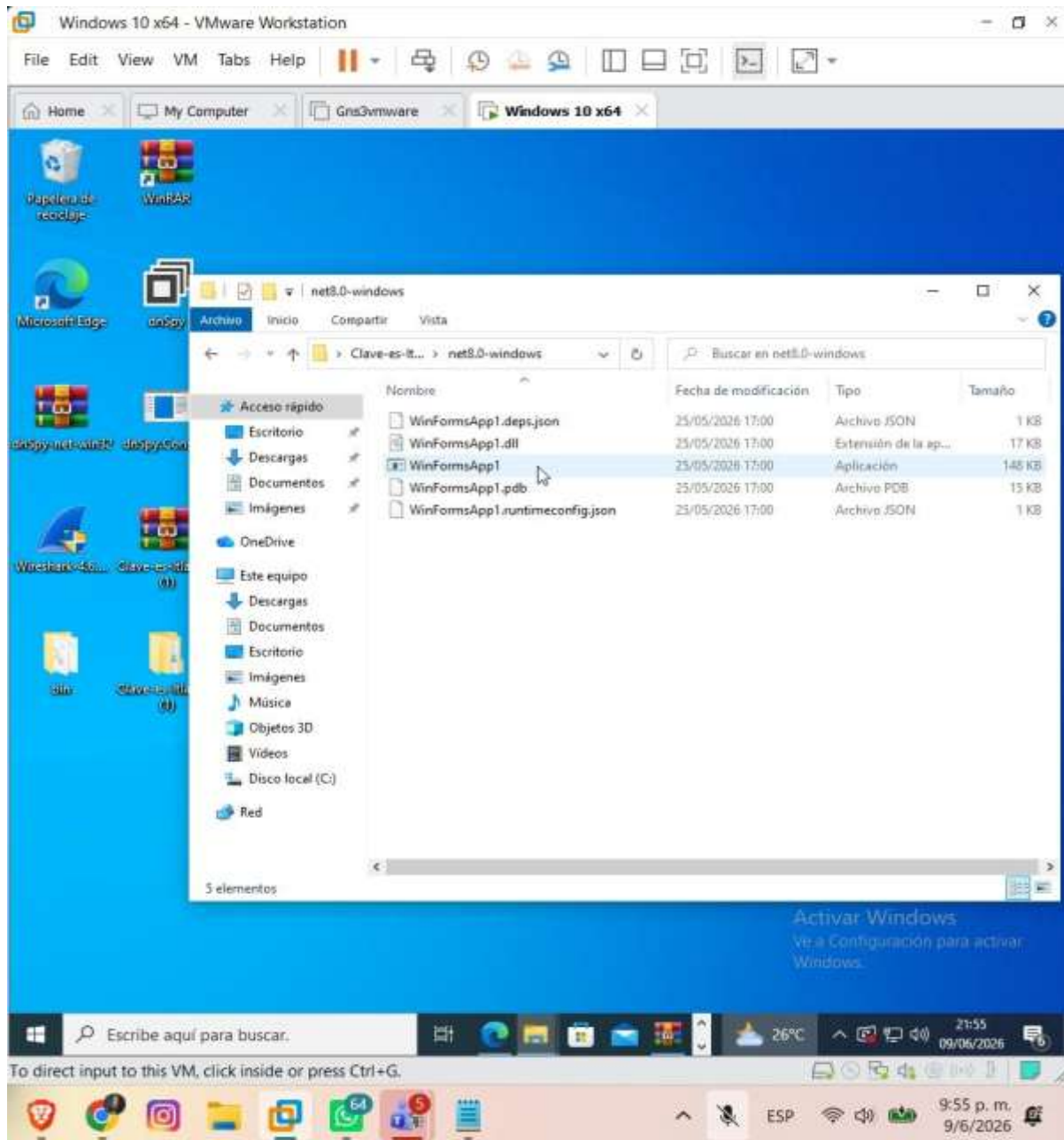


Figura 5 – Consola de administrador y ventana CyberOps en segundo plano

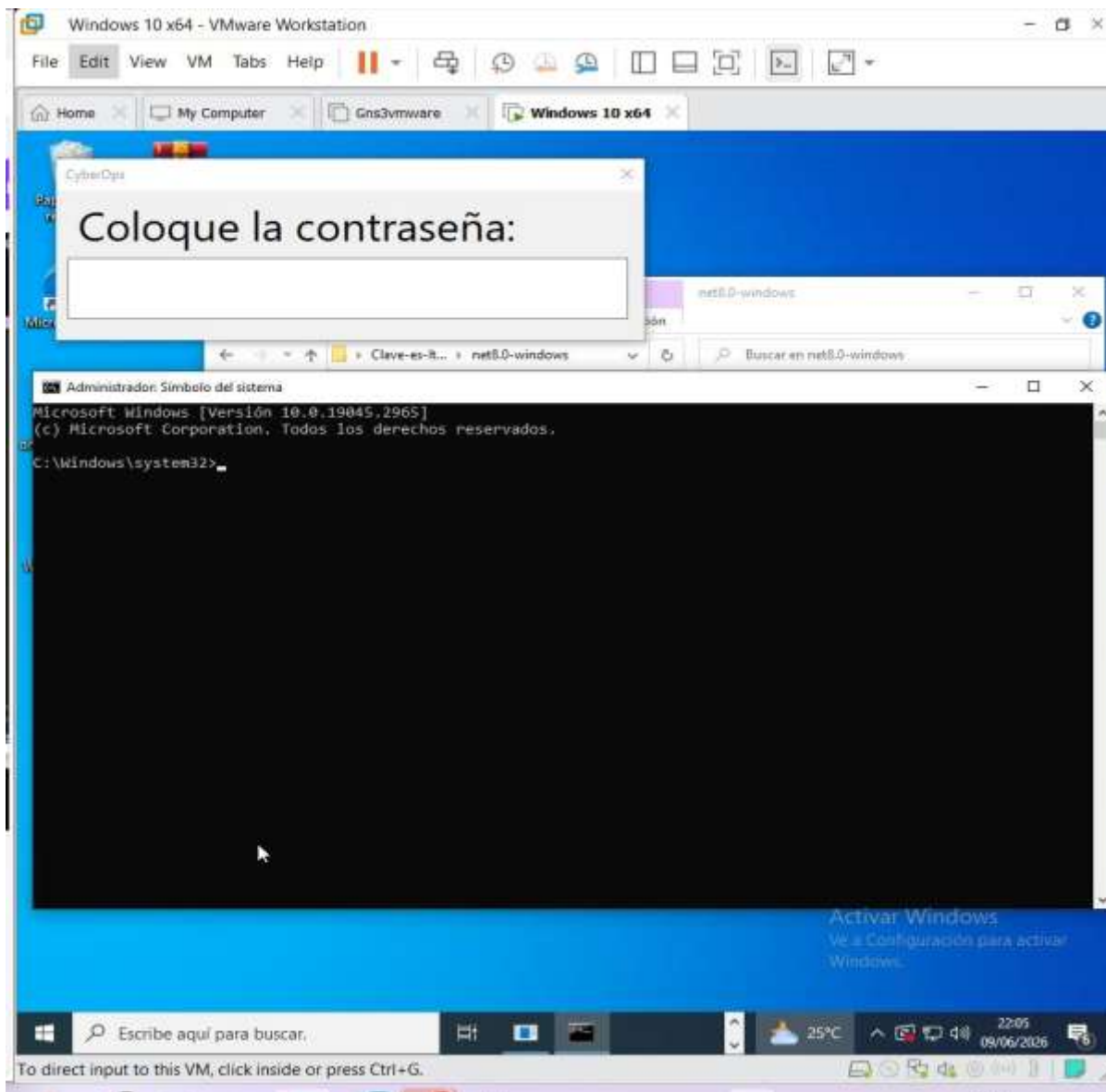


Figura 6 – Resultado de netstat mostrando puertos en estado LISTENING

Los puertos relevantes identificados fueron:

- Puerto 135 – RPC (PID 876)
- Puerto 445 – SMB (PID 4)
- Puerto 5040 (PID 1188)
- Puerto 7680 – WUDO (PID 5856)
- Puerto 49822 en 127.0.0.1 – Servicio local del programa (PID 7428)
- Puerto 42050 en [::1] – Servicio de pistas del programa

3.5 Conexión al Servicio de Pistas

Usando ncat, se conectó al puerto local 49822 en 127.0.0.1. El servidor respondió identificándose como "CyberOps Lab Hint Server by Rons" y proporcionó las siguientes pistas clave:

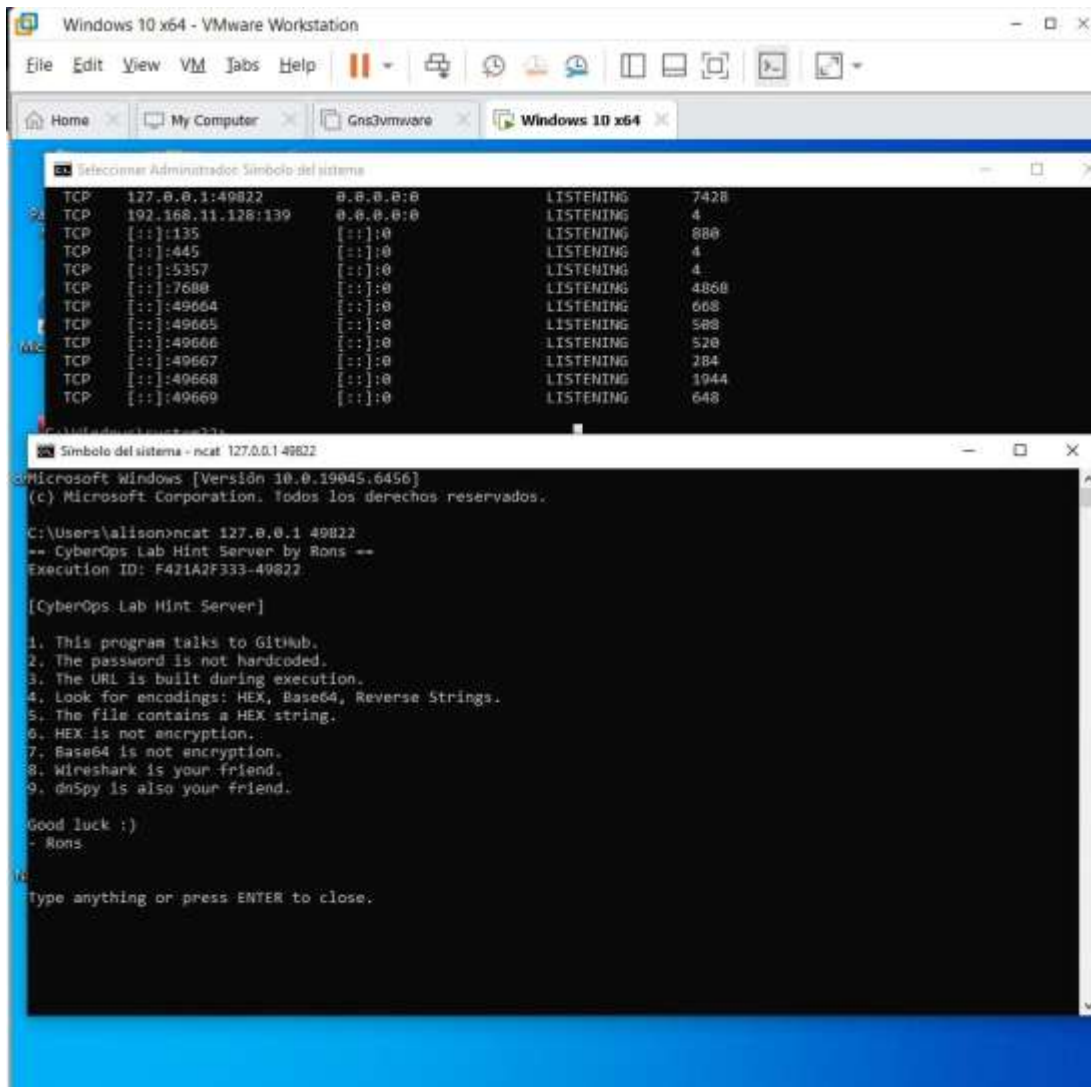


Figura 7 – Conexión exitosa al servicio de pistas en 127.0.0.1:49822

Las pistas recibidas fueron:

- 1. This program talks to GitHub.
- 2. The password is not hardcoded.
- 3. The URL is built during execution.
- 4. Look for encodings: HEX, Base64, Reverse Strings.
- 5. The file contains a HEX string.
- 6. HEX is not encryption.
- 7. Base64 is not encryption.
- 8. Wireshark is your friend.
- 9. dnSpy is also your friend.

3.6 Análisis de Tráfico HTTPS con Wireshark – Seguir Flujo TCP

En Wireshark se identificó un paquete TCP hacia el puerto 443 con SNI raw.githubusercontent.com. Se hizo clic derecho sobre el paquete y se seleccionó "Seguir > Flujo TCP" para inspeccionar el contenido de la comunicación.

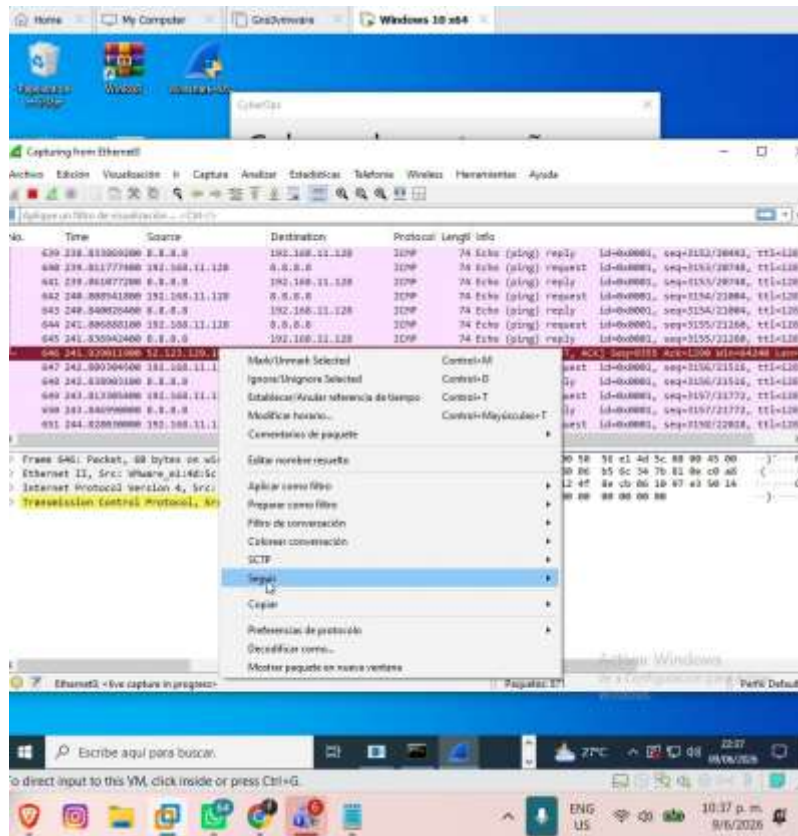


Figura 8 – Menú contextual en Wireshark: opción "Seguir" para seguir el flujo TCP

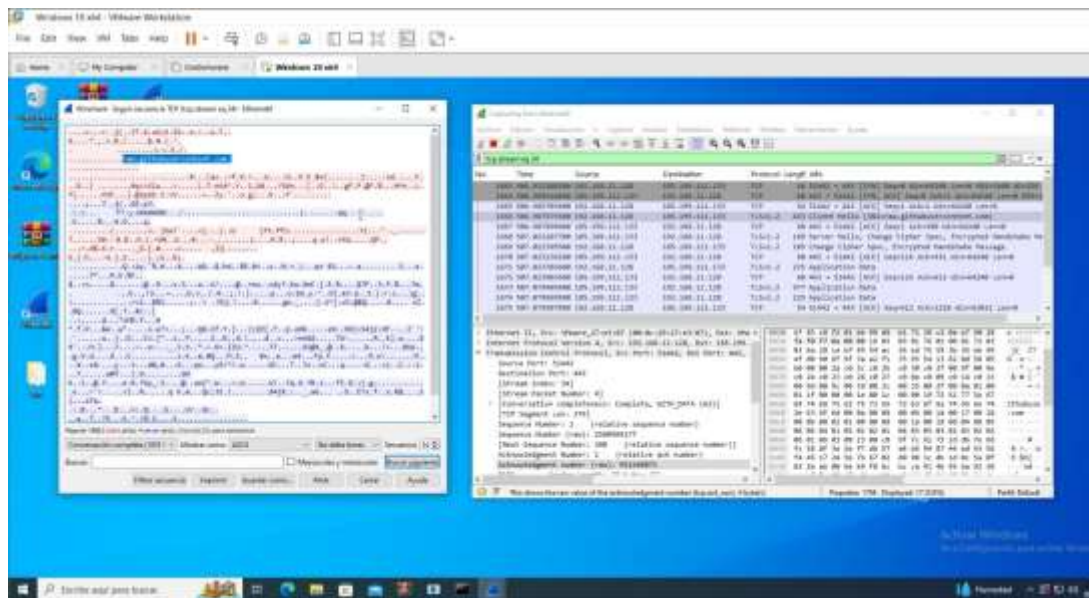


Figura 9 – Flujo TCP seguido, se observa la URL raw.githubusercontent.com en el stream

El flujo TCP reveló que el programa realiza una petición HTTPS a raw.githubusercontent.com para descargar un archivo. La URL visible en el stream es:
[https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/...](https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/)

3.7 Reversing con dnSpy

Se abrió dnSpy (versión 6.1.8 de 32 bits) y mediante Archivo > Abrir se cargó el archivo WinFormsApp1.dll de la carpeta net8.0-windows.

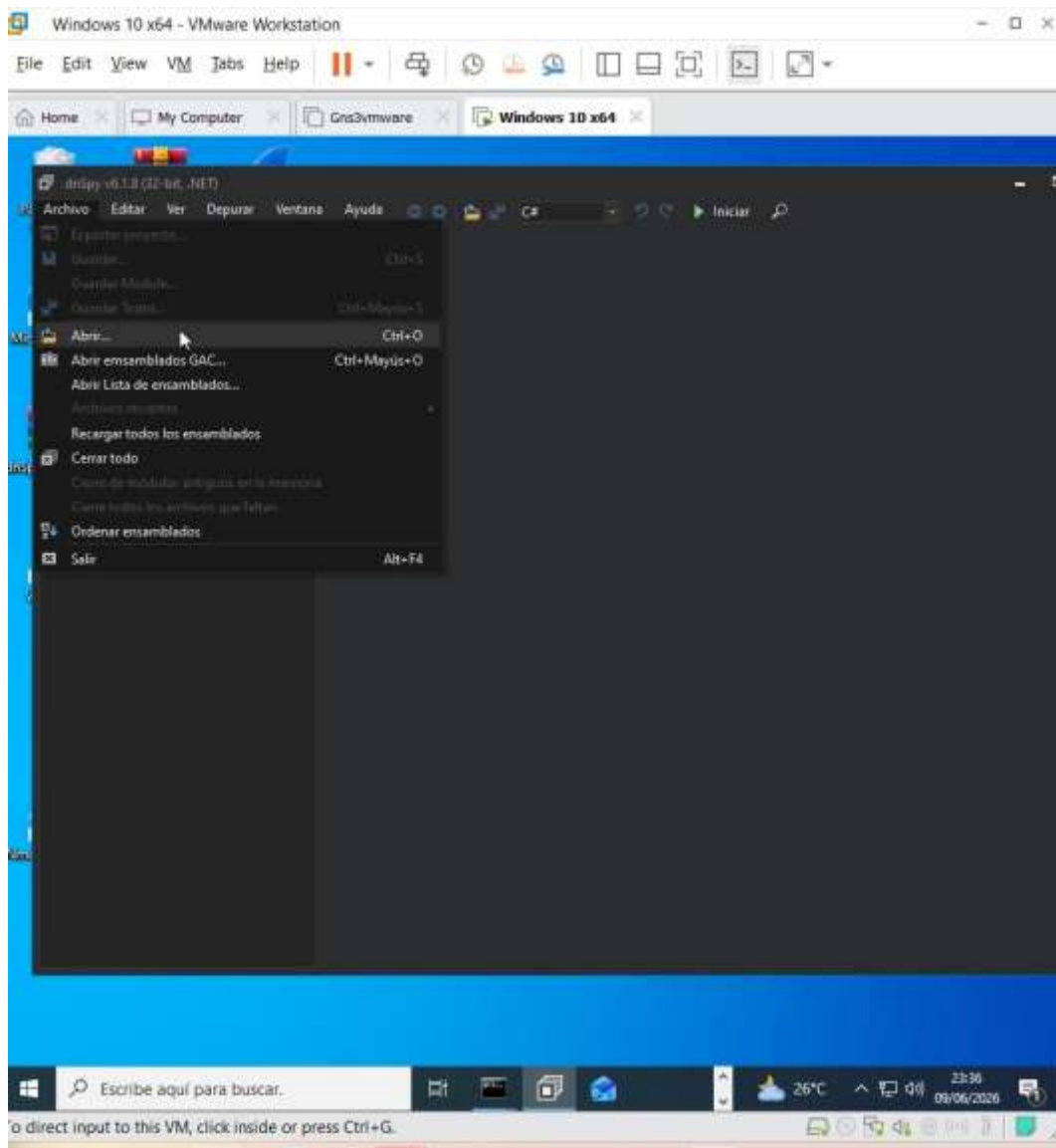


Figura 10 – dnSpy: menú Archivo > Abrir para cargar el ensamblado

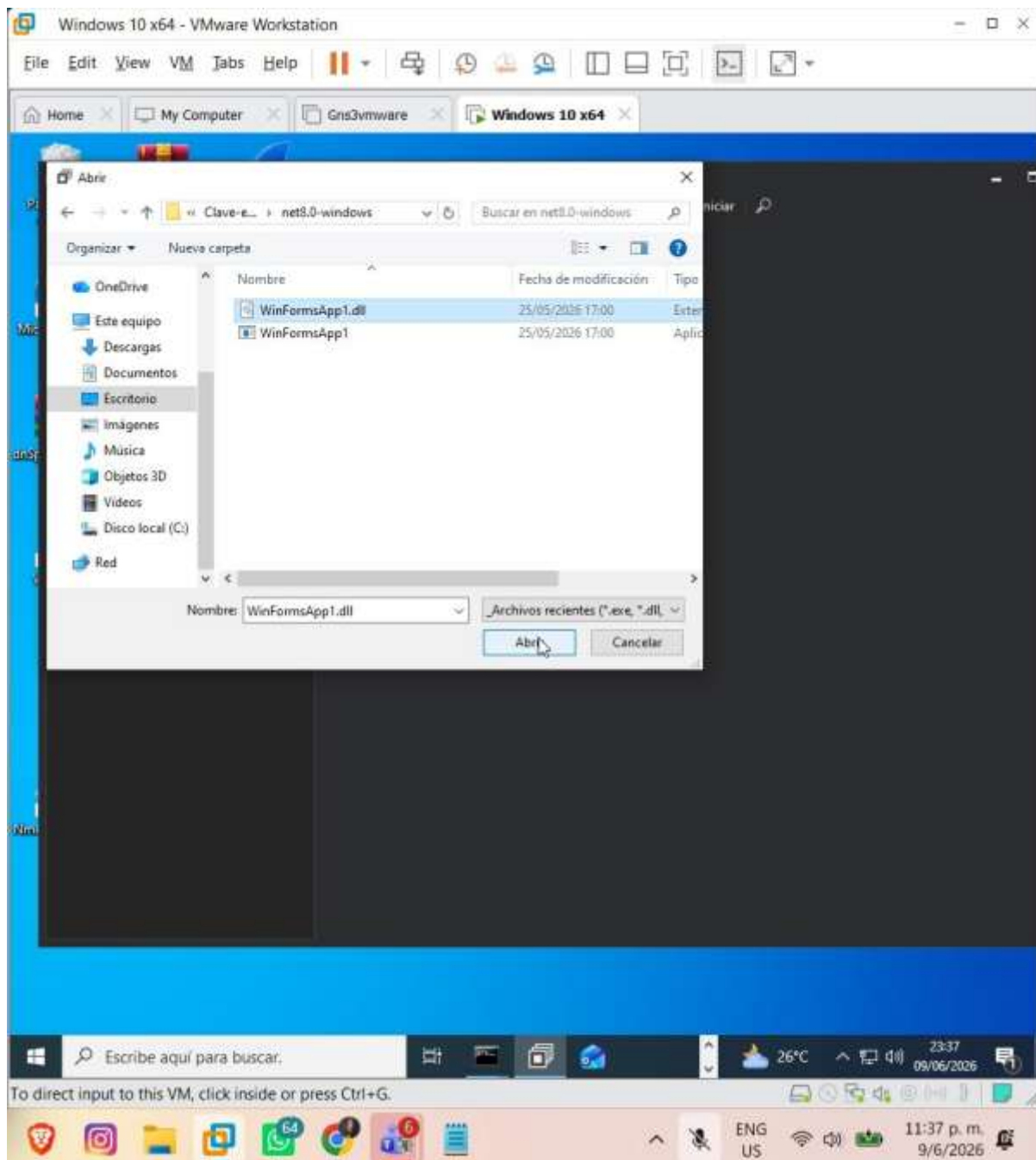


Figura 11 – Selección de WinFormsApp1.dll en el diálogo de apertura de dnSpy

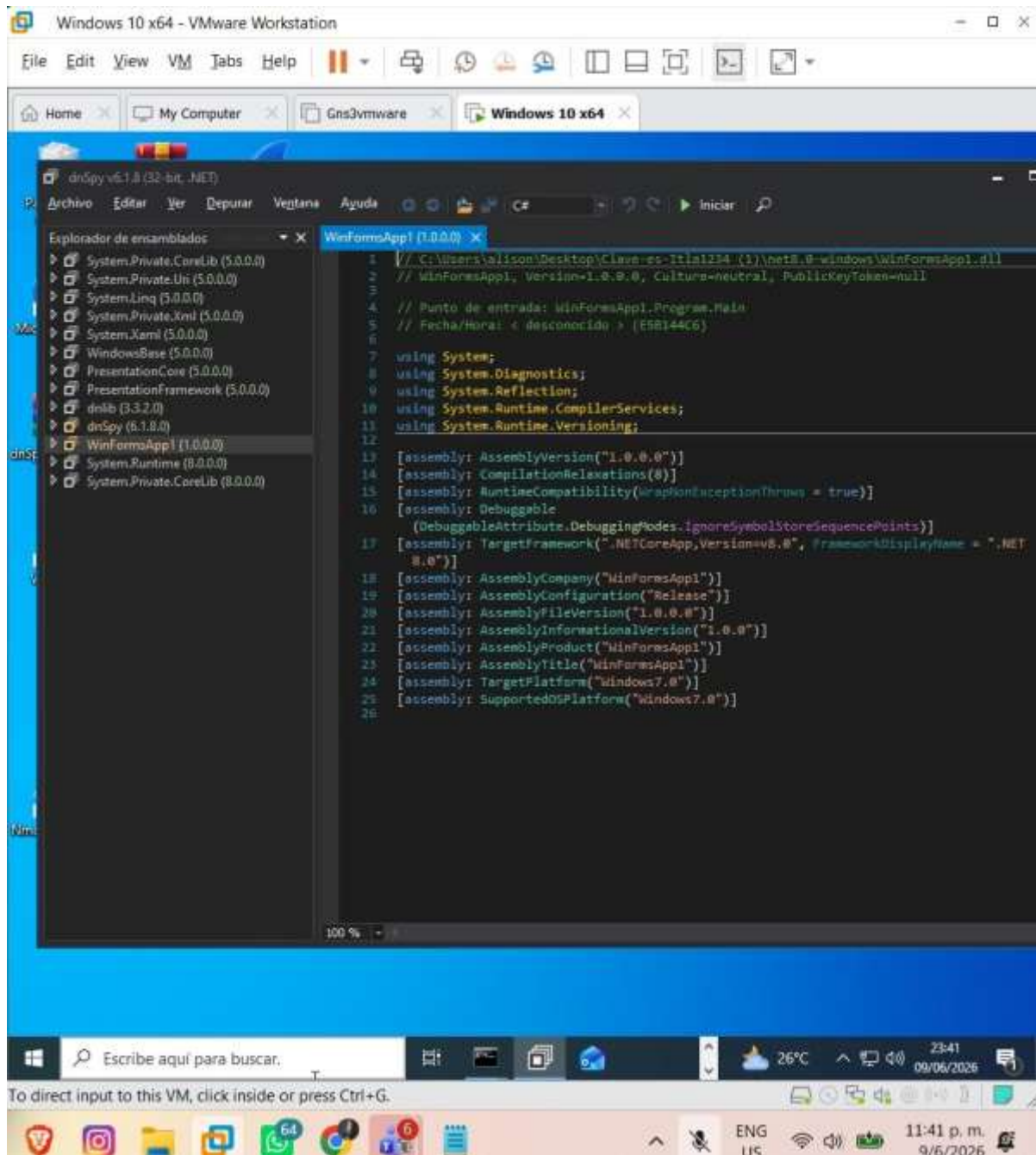


Figura 12 – Vista del ensamblado WinFormsApp1 (1.0.0.0) cargado en dnSpy

3.8 Análisis del Código Fuente Decompilado

Explorando el árbol de ensamblados en dnSpy, se accedió a WinFormsApp1 > Form1 y se inspeccionaron los métodos. En el método A25() se encontró una cadena Base64 muy larga que corresponde a la URL del archivo remoto en GitHub. El método usa `Encoding.UTF8.GetString(Convert.FromBase64String(a61))` para decodificarla y llama a `this.A31(a62)` para procesarla.

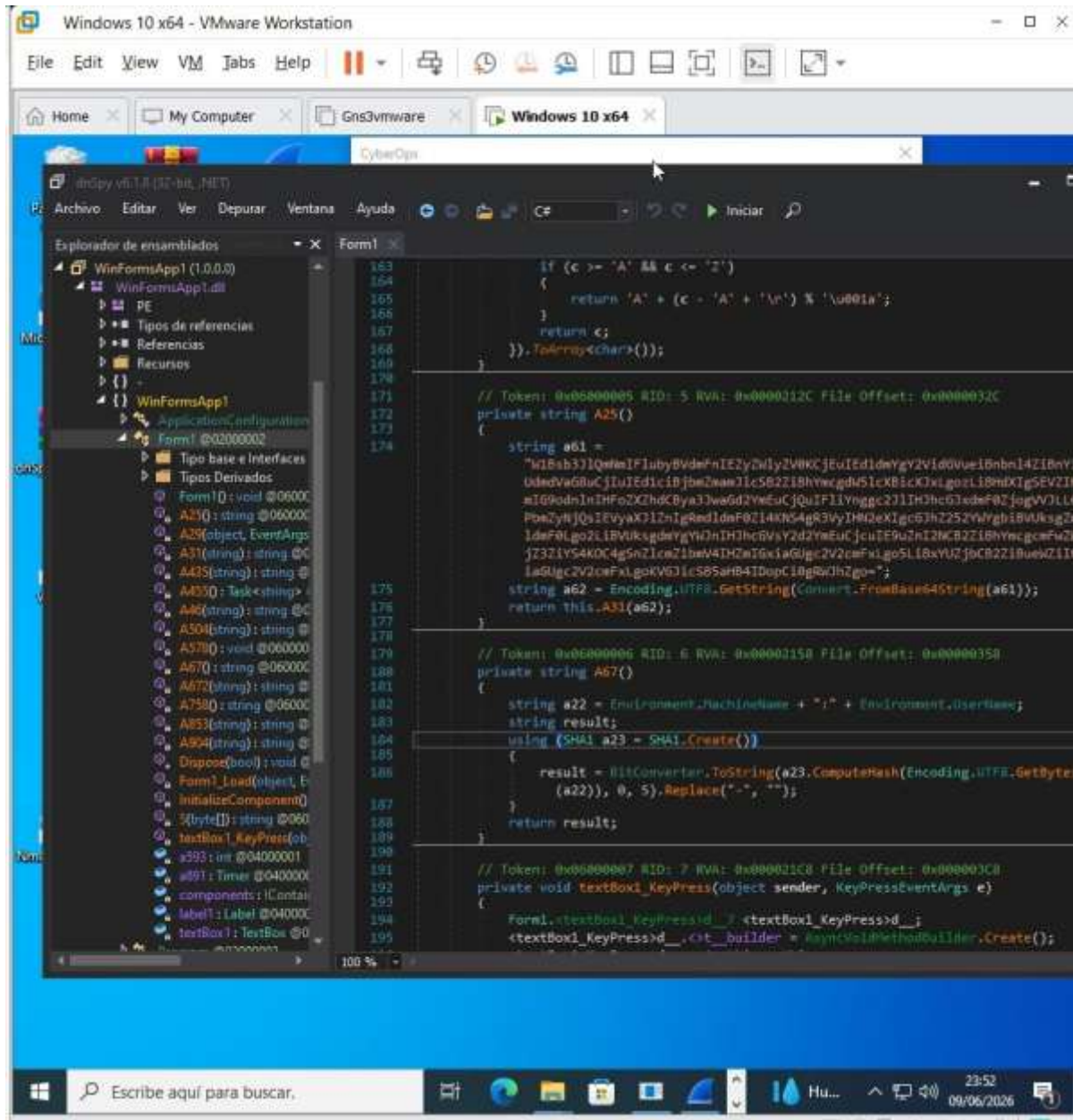


Figura 13 – Código del método A25() con la cadena Base64 y lógica de decodificación

El método A67() genera un hash SHA1 de la combinación MachineName:UserName para producir parte de la validación. También se encontró en los recursos hexadecimales del ensamblado la URL completa:

3.9 Vista Hexadecimal – URL en los Recursos

En la vista hexadecimal del ensamblado (sección de recursos/PE), se puede leer claramente el texto de la URL en formato ASCII embebido:

https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/0c46000d54d963148956df879eb518d384c5cf64/rons_sr.txt

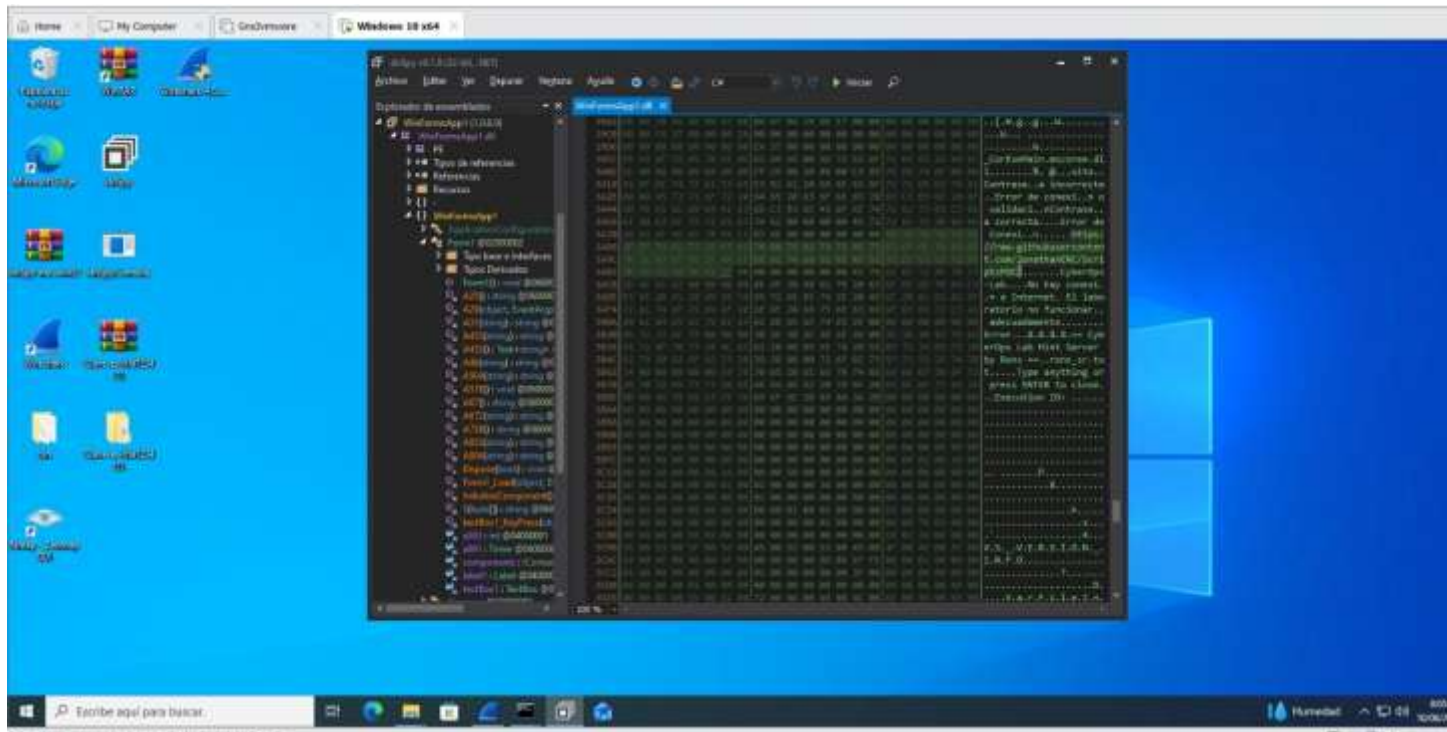


Figura 14 – Vista hexadecimal del ensamblado mostrando la URL de GitHub y mensajes del programa

También se pueden leer en el hex: mensajes como "Contraseña incorrecta", "Error de conexión", "Contraseña correcta", y el texto del servidor de pistas "CyberOps Lab Hint Server by Rons"

3.10 Decodificación de la Contraseña

Con la URL obtenida se accedió al archivo rons_sr.txt en GitHub Raw. Según las pistas del servidor y el análisis del código:

- El archivo contiene una cadena HEX.
- El HEX se decodifica a una cadena en Base64.
- El Base64 se decodifica para obtener la contraseña en texto plano (o en formato invertido/Reverse String).

La URL completa del recurso remoto encontrada fue:

https://raw.githubusercontent.com/JonathanERC/ScriptsPOC/0c46000d54d963148956df879eb518d384c5cf64/rons_sr.txt

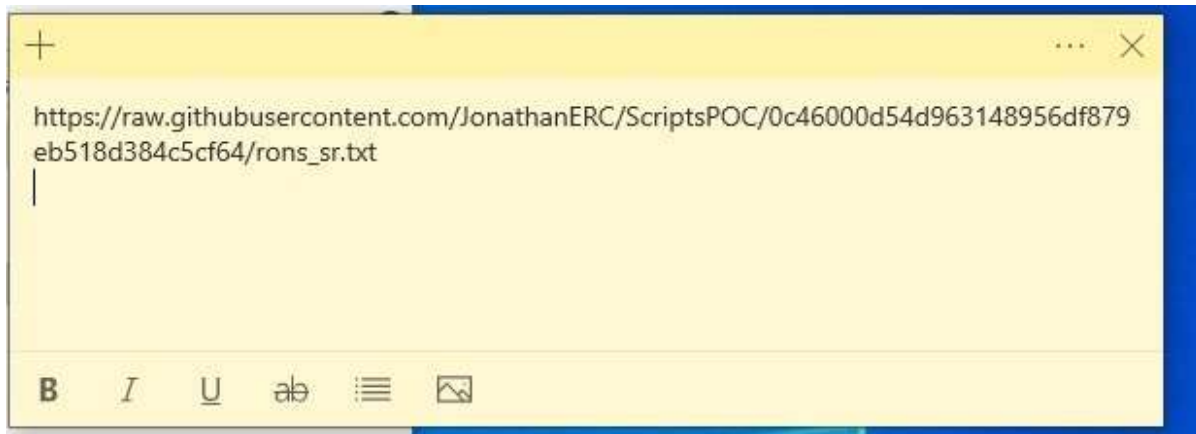
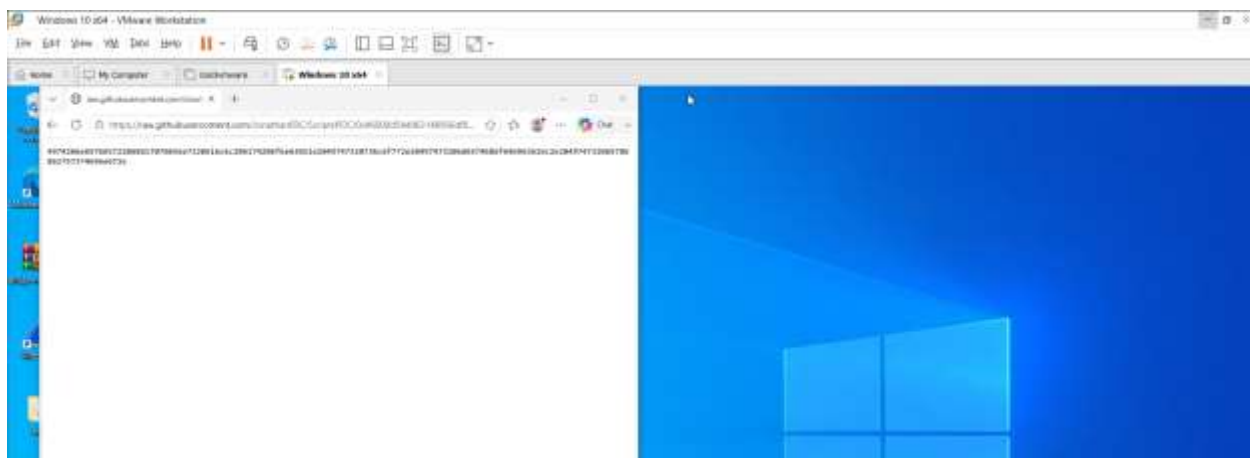


Figura 15 – Nota adhesiva con la URL completa del archivo remoto en GitHub Raw



Una ayudita XD



3.11 Contraseña Correcta

Tras completar el proceso de decodificación (HEX → Base64 → texto plano / Reverse String), se introdujo la contraseña en la aplicación y el programa mostró el mensaje "Contraseña correcta".

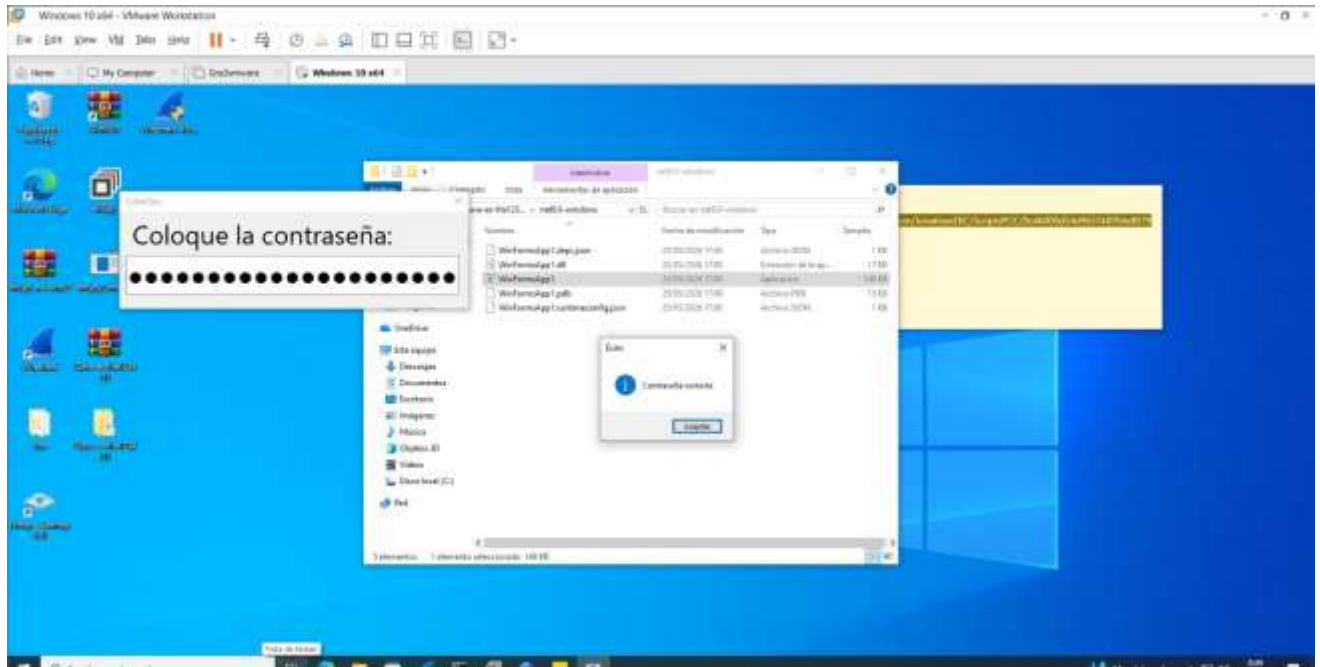


Figura 16 – Mensaje de éxito: "Contraseña correcta" al ingresar la contraseña decodificada

4. Respuestas a las Preguntas del Reporte

4.1 ¿Qué puertos utiliza el programa?

Durante la ejecución de WinFormsApp1, se identificaron los siguientes puertos:

- Puerto 49822 (TCP, 127.0.0.1) – Servidor local de pistas "CyberOps Lab Hint Server".
- Puerto 42050 (TCP, [::1]) – Servicio adicional local del programa.
- Puerto 443 (TCP, saliente) – Conexión HTTPS hacia raw.githubusercontent.com para descargar el archivo con la contraseña.
- Puerto 135, 445, 5040, 7680 – Puertos del sistema operativo Windows (no del programa).

4.2 ¿Cómo realiza el programa la prueba de conectividad?

El programa realiza la prueba de conectividad enviando paquetes ICMP Echo Request (ping) hacia la dirección 8.8.8.8 (Google Public DNS). Esto se verificó en Wireshark donde se observaron los paquetes ICMP request desde 192.168.11.128 hacia 8.8.8.8 y las respuestas correspondientes. Si hay conectividad, el programa procede a realizar la conexión HTTPS hacia GitHub para obtener la clave.

4.3 Enumera los encodings encontrados y dónde fueron utilizados

- HEX (Hexadecimal): El archivo remoto rons_sr.txt descargado desde GitHub contiene la contraseña codificada en formato hexadecimal.
- Base64: En el código fuente (método A25()), la URL de GitHub está codificada como una cadena Base64 muy larga. Se decodifica en tiempo de ejecución con Convert.FromBase64String(). El HEX del archivo remoto, al decodificarse, produce una cadena en Base64 que luego también se decodifica.
- Reverse String (cadena invertida): Como capa adicional de ofuscación, algunas cadenas están invertidas. El programa incluye lógica de rotación de caracteres (Caesar/ROT13 modificado) visible en el decompilado.
- SHA1: Se usa en el método A67() para generar un hash de MachineName:UserName, posiblemente como parte de la validación o identificación de ejecución.

4.4 ¿Qué tráfico genera el programa?

- ICMP Echo Request/Reply: Pings a 8.8.8.8 para verificar conectividad a Internet.
- TCP/TLS (HTTPS) hacia 185.199.111.133 puerto 443: Corresponde a raw.githubusercontent.com. El programa descarga el archivo rons_sr.txt que contiene la contraseña codificada.
- TCP local en 127.0.0.1:49822: El programa abre un servidor local de pistas al que se puede conectar con ncat para recibir hints del desafío.

4.5 ¿Cómo se conectó al servicio de pistas?

Se conectó al servicio de pistas usando ncat (netcat) desde una consola de comandos con el siguiente comando:

```
ncat 127.0.0.1 49822
```

Previamente se identificó el puerto 49822 escuchando en 127.0.0.1 mediante el comando netstat -ano | findstr LISTEN en la consola de administrador. Al conectarse, el servidor respondió con el mensaje de bienvenida "== CyberOps Lab Hint Server by Rons ==" y proporcionó 9 pistas fundamentales para resolver el reto.

5. Conclusiones

Esta práctica permitió aplicar de forma integral técnicas de análisis de malware/aplicaciones sospechosas en un entorno controlado. Se demostró cómo una aplicación puede:

- Ocultar su lógica mediante encodings en capas (HEX → Base64 → Reverse String).
- Verificar conectividad a internet mediante ICMP antes de operar.
- Descargar recursos remotos desde GitHub de forma dinámica usando una URL construida en tiempo de ejecución.
- Exponer servicios locales de comunicación (servidor de pistas en 127.0.0.1:49822).
- Ser analizada eficazmente mediante Wireshark (tráfico de red), netstat (puertos) y dnSpy (reversing del código .NET).

La combinación de análisis de tráfico con Wireshark y decompilación con dnSpy fue esencial para descubrir el mecanismo completo de validación de la contraseña, confirmando que el dominio del conocimiento de redes y herramientas de análisis es fundamental en CyberOps.